

SYSTÈME ET RÉSEAUX : PARTIEL 1

mars 2026 — Pierre Rousselin

Consignes et conseils :

- Les prototypes des appels système et de certaines fonctions sont rappelés en fin d'énoncé.
- Pour gagner du temps, sauf indication contraire, vous pourrez supposer que tous les appels système réussissent.
- Les inclusions de fichiers d'en-tête ne sont pas nécessaires non plus.
- Il est inutile de recopier l'énoncé. Lorsque vous devez compléter des programmes, il suffit d'indiquer clairement à quel endroit (numéro de ligne, corps de fonction...) vous ajoutez des instructions ou modifiez le programme.
- La clarté et la simplicité des solutions seront des éléments importants de l'évaluation.
- Le barème est donné à titre indicatif et est susceptible d'être modifié avant la fin du processus.

Exercice 1 : *parallel for* sur 12 points

Une certaine fonction est supposée donnée et sans effets de bord :

```
/* Retourne le résultat d'un long calcul associé à l'argument n */
double compute(int k);
```

1. Écrire un programme `notparallelfor.c` qui :

- met dans une variable `n` de type `int` la valeur entière associée à son unique argument de la ligne de commande, supposé composé de chiffres décimaux
- alloue sur le tas un tableau `results` pouvant stocker `n` valeurs de type `double`
- stocke dans `results[0]` la valeur de `compute(0)`, dans `results[1]` la valeur de `compute(1)`, ..., dans `results[n - 1]` la valeur de `compute(n - 1)`,
- puis, une fois tous les calculs faits, affiche le contenu du tableau, le libère et se termine avec un statut de sortie correspondant à un succès.
- Lorsqu'il n'y a pas un seul argument de la ligne de commande un message d'erreur est affichée sur la sortie appropriée et le processus prend fin avec un statut de sortie approprié.
- Même chose lorsque l'argument de la ligne de commande n'est pas interprétable comme un entier en décimal (utiliser la fonction `sscanf`).

2. Dans un programme nommé `parallelfor.c`, on suppose définie et entière la constante symbolique `NUM_THREADS` (qui vaut par exemple 8). Écrire les modifications à apporter au programme précédent pour faire exécuter les `n` calculs par `NUM_THREADS` *threads* avec les contraintes suivantes :

- le *thread* initial n'appelle pas lui-même la fonction `compute`; il crée `NUM_THREADS` *threads* et attend qu'ils aient terminé leurs calculs
- le thread numéro 0 calcule `results[0]`, `results[NUM_THREADS]`, `results[2 * NUM_THREADS]`, ...
- le thread numéro 1 calcule `results[1]`, `results[NUM_THREADS + 1]`, `results[2 * NUM_THREADS + 1]`, ...
- Il n'y a pas de variable globale mais il y a un type structuré `struct worker_arg` contenant les champs utiles aux *threads*; chaque *thread* reçoit l'adresse d'une variable de ce type allouée sur le tas, contenant les données dont elle a besoin pour calculer.
- Lorsque tous les calculs sont terminés, le *thread* initial affiche, comme dans la première version, les éléments du tableau `results`.
- Toute la mémoire allouée sur le tas doit être libérée.

--- * ---

Correction de l'exercice 1 :

1. Barème approximatif :

- test `argc`, interprétation `argv[1]` et gestion des erreurs sur 3 points
- `malloc` et `free` sur 1 point
- calculer, afficher sur 2 points

```
double compute(int n);

int main(int argc, char *argv[])
{
    if (argc != 2) {
        fprintf(stderr, "%s: Usage: %s N\n", argv[0], argv[0]);
        exit(1);
    }
    int n;
    if (sscanf(argv[1], "%d", &n) < 1) {
        fprintf(stderr, "%s: argument numérique attendu\n", argv[0]);
        exit(2);
    }
    double *results;
    results = malloc(n * sizeof(double));
    for (int i = 0; i < n; i++) {
        results[i] = compute(i);
    }

    for (int i = 0; i < n; i++) {
        printf("results[%d] = %lg\n", i, results[i]);
    }
    free(results);
    return 0;
}
```

2. Barème approximatif :

- `create` et `join` `N threads` sur 2 points
- définition du type `struct worker_arg` et passage des données sur 2 points
- définition de la `start_routine` commune sur 2 points

```
struct worker_arg {
    int from;
    int n;
    double *results;
};

/* worker_arg doit être l'adresse d'une struct worker_arg sur le tas */
void *worker(void *worker_arg);
int main(int argc, char *argv[])
{
    /* Même programme que précédemment */
    double *results = malloc(n * sizeof(double));
    pthread_t th[NUM_THREADS];
    for (int i = 0; i < NUM_THREADS; i++) {
        struct worker_arg *wa = malloc(sizeof(*wa));
        wa->from = i;
        wa->n = n;
        wa->results = results;
        pthread_create(th + i, NULL, worker, wa);
    }
}
```

```

    }

    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_join(th[i], NULL);
    }

    for (int i = 0; i < n; i++) {
        printf("results[%d] = %lg\n", i, results[i]);
    }
    free(results);
    return 0;
}
void *worker(void *worker_arg)
{
    struct worker_arg *wa = worker_arg;
    for (int i = wa->from; i < wa->n; i += NUM_THREADS) {
        wa->results[i] = compute(i);
    }
    free(wa);
    return NULL;
}

```

--- * ---

Exercice 2 : fork, wait, kill et exec (6 points)

On considère le programme suivant (les inclusions de fichiers d'en-tête ont été omises).

```

1  #define N_DFT 3
2  int main(int argc, char *argv[])
3  {
4      int n = argc > 1 ? atoi(argv[1]) : N_DFT;
5      if (n <= 0) {
6          printf("der !\n");
7          return 0;
8      }
9      printf("prem's !\n");
10     for (; n > 0; n--) {
11         pid_t p = getpid();
12         printf("lol\n");
13         if (fork() != 0) {
14             wait(NULL);
15             printf("plop\n");
16         }
17         kill(p, SIGKILL);
18     }
19     execlp("./a.out", "./a.out", "0", NULL);
20     printf("haha\n");
21     return 0;
22 }

```

On suppose que le binaire compilé à partir de ce programme s'appelle `a.out` et qu'il est présent dans le répertoire courant. Le signal `SIGKILL` met toujours fin au processus qui le reçoit.

- (4 points) Représenter l'arbre de création des processus lorsque le programme ci-dessus est exécuté sans argument. Y faire figurer les différents affichages et la mort de chaque processus.

2. (2 points) Donner, en fonction de n , le nombre d’affichage de chacune des chaînes de caractères présentes dans le programme lorsque celui-ci est exécuté par la commande `./a.out n` où n est un entier strictement positif.

--- * ---

Correction de l’exercice 2 :

1. Le processus initial affiche `prem's !` puis `lol`, fork puis est tué par son enfant.
Cet enfant affiche `lol`, puis fork et est tué par son enfant.
Ce nouvel enfant affiche `lol`, puis fork et est tué par son enfant.
Le dernier enfant, seul processus vivant à ce stade `exec` le même programme avec argument 0, on revient au début de `main`, il affiche `der !` puis meurt avec statut de sortie 0.
2. Il y a un seul affichage de `prem's`, par le processus initial. À chaque itération, un `lol` est affiché, un processus est créé tandis qu’un autre meurt. Il y a donc n affichages de `lol`. À la dernière itération, le dernier enfant sort de la boucle et `exec` le programme avec argument 0 donc il y a un affichage de `der` puis il meurt. La chaîne `haha` n’est jamais affichée.

--- * ---

Exercice 3 : Un petit calcul... (10 points)

On considère le programme suivant :

```
double calcul_a(double x, int n);
double calcul_b(double x, int n);
double calcul_c(double x, int n);
double calcul_d(double x, double y);
double calcul_e(double x);
double calcul_f(double x);

struct calcul {
    double x, a, b, c, d, e;
    int n;
};

int main(void)
{
    struct calcul calc;
    scanf("%lf %d", &(calc.x), &(calc.n));
    calc.a = calcul_a(calc.x, calc.n);
    calc.b = calcul_b(calc.x, calc.n);
    calc.c = calcul_c(calc.x, calc.n);
    calc.d = calcul_d(calc.a, calc.b);
    calc.e = calcul_e(calc.b + calc.c);
    double res = calcul_f(calc.e - calc.d);
    printf("Résultat : %g\n", res);
    return 0;
}
```

Les fonctions `calcul_a`, ... `calcul_f` sont supposées données et sans effets de bord.

1. (2 points) Donner le diagramme de dépendance entre les différents calculs.
2. (8 points) Sans modifier le type des fonctions `calcul_a`, ..., `calcul_f`, en utilisant des *threads* et éventuellement des sémaphores, changer le programme de façon à paralléliser au maximum les calculs, tout en respectant les dépendances. Le ou les sémaphores pourront être globaux mais pas les autres variables éventuelles. **La simplicité du programme sera prise en compte dans l’évaluation.**

--- * ---

Pour les correcteurs : Pour la première question, 2 points pour gratifier les étudiants qui sont entrés dans l'exercice. Pour la deuxième question, 8 points à la discrétion des correcteurs, prendre en compte :

- la création des threads
- le passage d'arguments à ces threads
- la déclaration et l'initialisation d'un ou plusieurs sémaphores
- la synchronisation correcte des tâches
- l'attente des threads avec `join`
- pour passer de 6 à 8, prendre en compte l'élégance et la simplicité de la solution.

Correction de l'exercice 3 :

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>

double calcul_a(double x, int n);
double calcul_b(double x, int n);
double calcul_c(double x, int n);
double calcul_d(double x, double y);
double calcul_e(double x);
double calcul_f(double x);

struct calcul {
    double x, a, b, c, d, e;
    int n;
};

void *f_b(void *arg);

sem_t b_fini;
int main(void)
{
    double x;
    int n;
    scanf("%lf %d", &x, &n);
    sem_init(&b_fini, 0, 0);
    pthread_t t_b, t_ce;
    struct calcul calc = { .x = x, .n = n };
    pthread_create(&t_b, NULL, f_b, &calc);
    pthread_create(&t_ce, NULL, f_b, &calc);
    calc.a = calcul_a(x, n);
    pthread_join(t_b, NULL);
    calc.d = calcul_d(calc.a, calc.b);
    pthread_join(t_ce, NULL);
    double f = calcul_f(calc.e - calc.d);
    printf("Résultat : %g\n", f);
    return 0;
}

double calcul_a(double x, int n) { return x + n; }
```

```

double calcul_b(double x, int n) { return x - n; }
double calcul_c(double x, int n) { return x * n; }
double calcul_d(double x, double y) { return x + y; }
double calcul_e(double x) { return x * x; }
double calcul_f(double x) { return x + x; }

void *f_b(void *arg)
{
    struct calcul *calc = arg;
    calc->b = calcul_b(calc->x, calc->n);
    sem_post(&b_fini);
    return NULL;
}

void *f_ce(void *arg)
{
    struct calcul *calc = arg;
    calc->c = calcul_c(calc->x, calc->n);
    sem_wait(&b_fini);
    calc->e = calcul_e(calc->b + calc->c);
    return NULL;
}

```

--- * ---

Exercice 4 : Le cuisinier et les mangeurs¹ (10 points)

Un cuisinier a invité NB_MANGEURS pour un dîner sans fin chez lui. Ce cuisinier a une marmite qui peut contenir N repas et il ne sait cuisiner que N repas à la fois.

Quand un mangeur veut manger, il va chercher un repas dans la marmite. Si la marmite est vide, il demande au cuisinier de la remplir.

Quand le cuisinier a fini de cuisiner, il attend qu'on l'appelle pour venir remplir la marmite avec ce qu'il a préparé.

Dans le code qui suit (inclusions de fichiers d'en-tête omises), la marmite est représentée par une pile bornée globale. Lorsqu'un mangeur se sert dans la marmite il dépile un repas et lorsque le cuisinier la remplit (toujours lorsqu'elle est vide), il empile N repas. Les fonctions **manger** et **cuisiner** sont données à titre d'exemple mais vous ne ferez aucune hypothèse sur la durée de ces tâches.

```

1  #define N 5
2  int marmite[N];
3  int nb_repas_marmite = 0;
4  void *cuisinier(void *arg);
5  void *mangeur(void *arg);
6  int cuisiner(void);
7  void remplir_marmite(int repas);
8  int se_servir_dans_marmite(void);
9  void manger(int repas);
10 #define NB_MANGEURS 7
11 /* ZONE D'AJOUT 1 */
12 int main(void)
13 {
14     pthread_t mangeurs[NB_MANGEURS], cuisto;
15     int i;
16     /* ZONE D'AJOUT 2 */

```

1. d'après un problème de Greg Andrews

```

17     pthread_create(&cuisto, NULL, cuisinier, NULL);
18     for (i = 0; i < NB_MANGEURS; i++)
19         pthread_create(&mangeurs[i], NULL, mangeur, NULL);
20     /* ZONE D'AJOUT 3 */
21     return 0;
22 }
23 int cuisinier(void) { return rand() % 10; }
24 void remplir_marmite(int repas)
25 {
26     int i;
27     for (i = 0; i < N; i++)
28         marmite[nb_repas_marmite++] = repas;
29 }
30 int se_servir_dans_marmite(void) { return marmite[--nb_repas_marmite]; }
31 void manger(int repas) { printf("Miam, quel bon %d !\n", repas); }
32 void *cuisinier(void *arg)
33 { /* À MODIFIER 1 */
34     for (;;) {
35         int repas = cuisinier();
36         remplir_marmite(repas);
37     }
38 }
39 void *mangeur(void *arg)
40 { /* À MODIFIER 2 */
41     for (;;) {
42         int repas = se_servir_dans_marmite();
43         manger(repas);
44     }
45 }

```

1. Ajouter les instructions permettant au *thread* initial d'attendre la fin de tous les *threads* créés (on fera « comme si » ces *threads* se terminent à un moment).
2. En ajoutant sémaphore(s) et/ou mutex et/ou variable(s) globale(s), synchroniser cuisinier et mangeurs comme indiqué ci-dessus. Les mutex et sémaphores créés devront être détruits (en faisant encore une fois « comme si » cuisinier et mangeurs se terminent à un moment).
Préserver au maximum le parallélisme (par exemple le cuisinier doit pouvoir cuisiner pendant que les mangeurs se servent, les mangeurs doivent pouvoir manger en même temps, ...).
Indication : seul un mangeur à la fois peut avoir accès à la marmite.

--- * ---

Pour les correcteurs :

- Première question sur 2 points.
- Deuxième question sur 8 points à répartir en :
 - mutex pour section critique (manipulation de la marmite)
 - sémaphore pour réveiller le cuisinier en cas de marmite vide
 - sémaphore pour réveiller les mangeurs lorsque la marmite est pleine
 - pas de deadlock
 - parallélisme possible entre mangeurs et entre les mangeurs et le cuisinier (qu'il puisse commencer à préparer les repas pendant que les mangeurs mangent)

Correction de l'exercice 4 :

```

#include <pthread.h>
#include <stdio.h>

```

```

#include <stdlib.h>
#include <semaphore.h>

#define N 5
int marmite[N];
int nb_repas_marmite = 0;

void *cuisinier(void *arg);
void *mangeur(void *arg);

int cuisiner(void);
void remplir_marmite(int repas);
int se_servir_dans_marmite(void);
void manger(int repas);

#define NB_MANGEURS 7

pthread_mutex_t acces_marmite;
sem_t marmite_vide;
sem_t marmite_pleine;

int main(void)
{
    pthread_t mangeurs[NB_MANGEURS], cuisto;
    int i;
    pthread_mutex_init(&acces_marmite, NULL);
    /* C'est un mangeur qui signale que la marmite est vide (d'où le 0) */
    sem_init(&marmite_vide, 0, 0);
    sem_init(&marmite_pleine, 0, 0);
    pthread_create(&cuisto, NULL, cuisiner, NULL);
    for (i = 0; i < NB_MANGEURS; i++)
        pthread_create(&mangeurs[i], NULL, mangeur, NULL);

    pthread_join(cuisto, NULL);
    for (i = 0; i < NB_MANGEURS; i++)
        pthread_join(mangeurs[i], NULL);
    pthread_mutex_destroy(&acces_marmite);
    sem_destroy(&marmite_vide);
    sem_destroy(&marmite_pleine);
    return 0;
}

int cuisiner(void) { return rand() % 10; }
void remplir_marmite(int repas)
{
    int i;
    for (i = 0; i < N; i++)
        marmite[nb_repas_marmite++] = repas;
}
int se_servir_dans_marmite(void) { return marmite[--nb_repas_marmite]; }
void manger(int repas) { printf("Miam, quel bon %d !\n", repas); }

void *cuisinier(void *arg)

```

```

{
    for (;;) {
        int repas = cuisiner();
        sem_wait(&marmite_vide);
        remplir_marmite(repas);
        sem_post(&marmite_pleine);
    }
}

void *mangeur(void *arg)
{
    for (;;) {
        pthread_mutex_lock(&acces_marmite);
        if (nb_repas_marmite == 0) {
            sem_post(&marmite_vide);
            sem_wait(&marmite_pleine);
        }
        int repas = se_servir_dans_marmite();
        pthread_mutex_unlock(&acces_marmite);
        manger(repas);
    }
}

--- * ---

```

Prototypes des fonctions et appels systèmes à utiliser

Certains arguments sont mis à NULL ou 0 car nous n'avons pas vu leurs rôles en cours.

```

/* Fonctionne comme scanf mais lit la chaîne str au lieu de l'entrée
   standard ; retourne le nombre d'entrées correctement associées dans str */
int sscanf(const char *restrict str, const char *restrict format, ...);
void *malloc(size_t n);
void free(void *p);
pid_t fork(void);
pid_t wait(int *wstatus);
int execlp(const char *file, const char *arg, ..., NULL);
int kill(pid_t pid, int sig);
int pthread_create(pthread_t *thread, NULL,
                  void *(*start_routine) (void *), void *arg);
int pthread_join(pthread_t thread, void **retval);
int pthread_mutex_init(pthread_mutex_t *mutex, NULL);
int pthread_mutex_lock(pthread_mutex_t *mutex);
int pthread_mutex_unlock(pthread_mutex_t *mutex);
int pthread_mutex_destroy(pthread_mutex_t *mutex);
int sem_init(sem_t *sem, 0, unsigned int valeur);
int sem_wait(sem_t *sem);
int sem_post(sem_t *sem);
int sem_destroy(sem_t *sem);

```